Unit 10 Exercises

4SC000 Optimal Control and Reinforcement Learning

Duarte Antunes

These exercises accompany Unit 10 and its slides.

> ⚠ Note on the transition matrices
>
> In Problems 10.3 and 10.6 the transition matrices are defined by
>
> $$P_{ji}(\ell) = \text{Prob}[x_{k+1} = j \mid x_k = i,\ u_k = \ell],$$
>
> so the entry in row j, column i is the probability of moving **from** i **to** j. Each **column** of $P(\ell)$ therefore sums to one. This is the transpose of the convention $p_{ij}(u)$ used in the notes and the slides. When you write the Bellman equation
>
> $$J(i) = \min_u\ g(i,u) + \alpha \sum_j p_{ij}(u) J(j),$$
>
> the sum $\sum_j p_{ij}(u)J(j)$ is the inner product of the i-th **column** of $P(u)$ with the vector of values – not the i-th row. Getting this backwards is the commonest way to lose these problems.

Core exercises

Problem 10.1 Consider the following finite horizon optimal control problem

$$\min_{u_k = \mu_k(x_k)} \mathbb{E}\left[\sum_{k=0}^{2} g(x_k, u_k) + g_3(x_3)\right]$$

$$x_{k+1} = \min(\max(x_k + u_k + w_k, 0), 3)$$

with $g(x_k, u_k) = x_k + u_k$ and $g_3(x_3) = x_3$, and $x_k \in \{0,1,2,3\}$, $u_k \in \{-1,0,1\}$, $w_k \in \{-1,0,1\}$, $\text{Prob}[w_k = -1] = \text{Prob}[w_k = 0] = \text{Prob}[w_k = 1] = \frac{1}{3}$.

(i) Write it as a shortest path problem.

(ii) Write the Bellman equation for the shortest path problem and show that the solution of the Bellman equation provides the optimal cost-to-go for every $k \in \{0,1,2\}$ for the original problem.

> ℹ Solution
>
> **(i)** Augment the state with the clock. Let $\bar{x}_k = (x_{1,k}, x_{2,k}) = (x_k, k)$ for $k \in \{0,1,2,3\}$, so that $\bar{x}_k \in \{0,1,2,3\} \times \{0,1,2,3\}$. (One could index $\bar{x}_k$ by a single $\hat{x}_k \in \{1,2,\dots,4\times 4\}$ to put the state in canonical form; nothing below depends on it.) The cost and the model become

$$\min_{u_k=\mu(\bar{x}_k)} \mathbb{E}\left[\sum_{k=0}^{\infty} \bar{g}(\bar{x}_k, u_k, w_k)\right]$$

with

$$\bar{g}(\bar{x}_k, u_k, w_k) = \begin{cases} x_{1,k} + u_k & \text{if } k \in \{0,1\} \\ x_{1,2} + u_2 + \min(\max(x_{1,2} + u_2 + w_2, 0), 3) & \text{if } k = 2 \\ 0 & \text{if } k \geq 3 \end{cases}$$

$$\bar{x}_{k+1} = (x_{1,k+1}, x_{2,k+1}) = (\min(\max(x_{1,k} + u_k + w_k, 0), 3),\ x_{2,k} + 1) := \bar{f}(\bar{x}_k, u_k, w_k).$$

The states with $x_{2,k} \geq 3$ are absorbing and cost-free, so this is a shortest path problem. Note that the terminal cost g_3 has been folded into the stage cost at $k = 2$: at that stage the running cost accounts both for the decision taken and for the state it leads to.

(ii) The Bellman equation for the shortest path problem is

$$\bar{J}(\bar{x}) = \min_{u\in\{-1,0,1\}} \mathbb{E}_{w\in\{-1,0,1\}}[\bar{g}(\bar{x}, u, w) + \bar{J}(\bar{f}(\bar{x}, u, w))].$$

Write $J_k(x_k) = \bar{J}(\bar{x}_k) = J_{x_{2,k}}(x_{1,k})$, i.e. use the clock component as a subscript. For states $\bar{x}_0 = (x_{1,0}, 0)$,

$$\begin{aligned} J_0(x_0) = \bar{J}(\bar{x}_0) &= \min_{u\in\{-1,0,1\}} \mathbb{E}_{w_0}[\bar{g}(\bar{x}_0, u, w) + \bar{J}(\min(\max(x_{1,0} + u + w, 0), 3),\ \underbrace{x_{2,0}}_{=0} + 1)] \\ &= \min_{u_0\in\{-1,0,1\}} x_0 + u_0 + \mathbb{E}_{w_0}[J_1(\min(\max(x_0 + u_0 + w_0, 0), 3))], \end{aligned}$$

for states $\bar{x}_1 = (x_1, 1)$,

$$J_1(x_1) = \min_{u_1\in\{-1,0,1\}} x_1 + u_1 + \mathbb{E}_{w_1}[J_2(\min(\max(x_1 + u_1 + w_1, 0), 3))],$$

and for states $\bar{x}_2 = (x_2, 2)$,

$$J_2(x_2) = \min_{u_2\in\{-1,0,1\}} \mathbb{E}_{w_2}[x_2 + u_2 + \min(\max(x_2 + u_2 + w_2, 0), 3) + J_3(\min(\max(x_2 + u_2 + w_2, 0), 3))].$$

Since $\bar{g}((i,k), u) = 0$ for $k \geq 3$, we have $\bar{J}((i,3)) = \bar{J}((i,4)) = \cdots = 0$, that is $J_3(i) = J_4(i) = \cdots = 0$ for every i. Substituting $J_3 \equiv 0$ into the last equation leaves exactly the dynamic programming recursion of the original finite horizon problem, run backwards from the terminal cost $g_3(x_3) = x_3$. So the solution of the single, time-invariant Bellman equation provides the optimal cost-to-go J_k for every $k \in \{0,1,2\}$ of the original problem – the time indices that were stages there are state components here.

Problem 10.2 Consider the following discounted cost optimal control problem

$$\min_{u_k=\mu_k(x_k)} \sum_{k=0}^{\infty} \mathbb{E}\left[\alpha^k g(x_k, u_k)\right]$$

$$x_{k+1} = \min(\max(x_k + u_k + w_k, 0), 3)$$

with $\alpha = 0.6$, $g(x_k, u_k) = x_k + u_k$ and $x_k \in \{0,1,2,3\}$, $u_k \in \{-1,0,1\}$, $w_k \in \{-1,0,1\}$, $\text{Prob}[w_k = -1] = \text{Prob}[w_k = 0] = \text{Prob}[w_k = 1] = \frac{1}{3}$.

(i) Write it as a shortest path problem.

(ii) Write the Bellman equation for the shortest path problem and show that it coincides with the Bellman equation for the original discounted cost problem.

Solution

(i) Follow the twin construction: multiply all transition probabilities by $\alpha = 0.6$ and route the remaining mass to a terminal state. Equivalently, enlarge the disturbance alphabet and set

$$\text{Prob}[w_k = -1] = \text{Prob}[w_k = 0] = \text{Prob}[w_k = 1] = \frac{\alpha}{3}, \qquad \text{Prob}[w_k = 2] = 1 - \alpha,$$

with the convention that $w_k = 2$ sends the state to a terminal state t:

$$x_{k+1} = \bar{f}(x_k, u_k, w_k) = \begin{cases} \min(\max(x_k + u_k + w_k, 0), 3) & \text{if } w_k \in \{-1, 0, 1\} \\ t & \text{if } w_k = 2. \end{cases}$$

The equivalent shortest path problem is then

$$\min_{u_k = \mu_k(x_k)} \sum_{k=0}^{\infty} \mathbb{E}\left[\bar{g}(x_k, u_k)\right], \qquad \bar{g}(x_k, u_k) = \begin{cases} x_k + u_k & \text{if } x_k \neq t \\ 0 & \text{if } x_k = t, \end{cases}$$

with terminal state t, and $x_k \in \{0, 1, 2, 3, t\}$, $u_k \in \{-1, 0, 1\}$, $w_k \in \{-1, 0, 1, 2\}$.
(ii) The Bellman equation for the shortest path problem is

$$J(i) = \min_{u \in \{-1,0,1\}} \bar{g}(i, u) + \mathbb{E}_{w \in \{-1,0,1,2\}}[J(\bar{f}(i, u, w))], \qquad i \in \{0, 1, 2, 3, t\}.$$

Since $\bar{g}(t, \cdot) = 0$ we have $J(t) = 0$, so the $w = 2$ branch contributes nothing to the expectation and

$$\begin{aligned} \mathbb{E}_{w \in \{-1,0,1,2\}}[J(\bar{f}(i, u, w))] &= \mathbb{E}_{w \in \{-1,0,1\}}[J(\bar{f}(i, u, w))] \\ &= \tfrac{\alpha}{3} J(\min(\max(i + u - 1, 0), 3)) + \tfrac{\alpha}{3} J(\min(\max(i + u, 0), 3)) + \tfrac{\alpha}{3} J(\min(\max(i + u + 1, 0), 3)). \end{aligned}$$

Hence, for $i \in \{0, 1, 2, 3\}$,

$$J(i) = \min_{u \in \{-1,0,1\}} g(i, u) + \alpha \underbrace{\left(\tfrac{1}{3} J(\min(\max(i + u - 1, 0), 3)) + \tfrac{1}{3} J(\min(\max(i + u, 0), 3)) + \tfrac{1}{3} J(\min(\max(i + u + 1, 0), 3))\right)}_{= \mathbb{E}_{w \in \{-1,0,1\}}[J(\min(\max(i+u+w,0),3))]},$$

which is exactly the Bellman equation for the discounted cost problem. The discount factor has reappeared not as a weight on future costs but as the probability α of *not* terminating – which is the content of the twin construction.

Problem 10.3 Consider the following shortest path optimal control problem

$$\min_{u_k = \mu_k(x_k)} \sum_{k=0}^{\infty} \mathbb{E}\left[g(x_k, u_k)\right], \qquad \text{Prob}[x_{k+1} = j \mid x_k = i,\ u_k = \ell] = P_{ji}(\ell),$$

for $i, j \in \{1, 2, 3\}$ and $\ell \in \{1, 2\}$, where $g(i, \ell) = 1$ for every $i \in \{1, 2\}$ and $\ell \in \{1, 2\}$, and $g(3, \ell) = 0$ for every $\ell \in \{1, 2\}$, and

$$P(1) = \begin{bmatrix} 0.1 & 0.5 & 0 \\ 0.9 & 0 & 0 \\ 0 & 0.5 & 1 \end{bmatrix}, \qquad P(2) = \begin{bmatrix} 0.9 & 0.9 & 0 \\ 0.1 & 0 & 0 \\ 0 & 0.1 & 1 \end{bmatrix}.$$

The initial condition is either $x_0 = 1$ or $x_0 = 2$, and the terminal condition is $x_k = 3$. The optimal policy is $\mu(1) = 1$, $\mu(2) = 1$, $\mu(3) = -$ (no control action is needed). Compute the optimal Q-function.

Solution

Start from the value function, using the Bellman equation with the optimal policy already supplied (so the min can be dropped and the known minimiser substituted):

$$J(1) = \min_{u\in\{1,2\}} g(1,u) + \sum_j p_{1j}(u)J(j) = g(1,1) + \sum_j P_{j1}(1)J(j) = 1 + 0.1J(1) + 0.9J(2)$$

$$J(2) = \min_{u\in\{1,2\}} g(2,u) + \sum_j p_{2j}(u)J(j) = g(2,1) + \sum_j P_{j2}(1)J(j) = 1 + 0.5J(1) + 0.5J(3)$$

$$J(3) = 0$$

(state 3 is terminal and free). Substituting the second equation into the first gives $J(1) = 1.9 + 0.55J(1)$, so

$$J(1) = \frac{1.9}{0.45} = 4.2222, \qquad J(2) = 1 + 0.5(4.2222) = 3.1111, \qquad J(3) = 0.$$

The optimal Q-function is $Q(i,u) = g(i,u) + \sum_j P_{ji}(u)J(j)$, i.e. the cost of taking u once and behaving optimally thereafter. Reading the columns of $P(1)$ and $P(2)$:

$$\begin{aligned} Q(1,1) &= 1 + 0.1J(1) + 0.9J(2) = 4.2222 & Q(1,2) &= 1 + 0.9J(1) + 0.1J(2) = 5.1111 \\ Q(2,1) &= 1 + 0.5J(1) + 0.5J(3) = 3.1111 & Q(2,2) &= 1 + 0.9J(1) + 0.1J(3) = 4.8000 \\ Q(3,1) &= 0 & Q(3,2) &= 0 \end{aligned}$$

The optimal policy is $\mu(i) = \arg\min_{u\in U(i)} Q(i,u)$. At state 1, $Q(1,1) < Q(1,2)$; at state 2, $Q(2,1) < Q(2,2)$; and $Q(3,1) = Q(3,2) = 0$, so no action matters there. This recovers exactly the policy given in the problem statement, which is the consistency check worth making: the Q-function was computed *assuming* that policy, so if its own minimisers disagreed with it, the assumed policy would not have been optimal.

Observe also that $Q(i,\mu(i)) = J(i)$ for each i – taking the optimal action and then behaving optimally is simply behaving optimally.

(The source solutions manual gives $Q(1,2) = 5.1109$ and $Q(2,2) = 4.7998$; the exact values are 5.1111 and 4.8000. The difference is rounding in the intermediate value of $J(1)$ and changes nothing.)

Problem 10.4 Consider an inventory control problem where the number of items of a given product over h stages is described by

$$x_{k+1} = \max\{x_k + u_k - d_k, 0\}, \qquad k \in \{0,\dots,h-1\},$$

where $x_k \in \{0,\dots,N\}$, $u_k \in U_k(x_k) := \{0,\dots,N - x_k\}$ and d_k denote the number of items, the supply and the demand at time k, and N denotes the capacity. The objective is to minimise

$$\sum_{k=0}^{h-1} \left(c_1(x_k) + c_2(u_k) - p\min\{x_k + u_k, d_k\}\right) + g_h(x_h),$$

where $c_1(i) = i$ is the storage cost,

$$c_2(j) = \begin{cases} 5j + 0.9 & \text{if } j > 0 \\ 0 & \text{if } j = 0, \end{cases}$$

is the cost of ordering j items, $p = 9$ is the selling price per item, and $g_h(i) = -4i$ is the terminal cost. Suppose $h = 3$, $N = 2$, and that the demand is uncertain at every stage, with $\text{Prob}[d_k = 0] = \text{Prob}[d_k = 1] = \text{Prob}[d_k = 2] = 1/3$.

Suppose Q-learning is used to obtain the optimal policy. The estimate at iteration ℓ — the iterations coinciding with the successive state/control pairs visited — for state $i \in \{0, 1, \dots, N\}$ at stage $k \in \{0, 1, \dots, h-1\}$ using control $u \in \{0, \dots, N-i\}$ is denoted $Q_\ell(k, i, u)$. A random policy with random initial conditions generates the trajectories summarised in the table below. All estimates are initialised to zero, $Q_1(k, i, u) = 0$ for every k, i, u.

ℓ	1	2	3	4	5	6	7	8	9	10
k	0	1	2	3	0	1	2	3	0	1
x_k	1	0	2	2	0	0	1	0	0	1
u_k	1	2	0	–	1	2	1	–	1	1
d_k	2	0	0	–	1	1	2	–	0	1

Provide the estimates after $L = 10$ iterations, $Q_{11}(k, i, u)$ for every k, i, u. Assume the step sizes are constant, $\gamma^\ell = 0.5$, in the update equations

$$Q_{\ell+1}(k, x_k, u_k) = (1-\gamma^\ell) Q_\ell(k, x_k, u_k) + \gamma^\ell \Big(g(x_k, u_k, d_k) + \min_{v \in U_{k+1}(x_{k+1})} Q_\ell(k+1, x_{k+1}, v) \Big), \quad k \neq h-1,$$

$$Q_{\ell+1}(h-1, x_{h-1}, u_{h-1}) = (1-\gamma^\ell) Q_\ell(h-1, x_{h-1}, u_{h-1}) + \gamma^\ell (g(x_{h-1}, u_{h-1}, d_{h-1}) + g_h(x_h)),$$

where $g(x_k, u_k, d_k) = c_1(x_k) + c_2(u_k) - p \min\{x_k + u_k, d_k\}$.

Solution

The running cost is $g(x_k, u_k, d_k) = x_k + c_2(u_k) - 9 \min\{x_k + u_k, d_k\}$. Two things to read off the table before starting. The rows with $k = 3 = h$ carry no control: the process has reached the terminal stage, there is nothing to decide and no pair (k, i, u) to update. And stage $k = 2 = h - 1$ uses the *second* update equation, where the bootstrap is the known terminal cost g_h rather than a stored estimate. Taking the ten iterations in turn:

Table 1: The Q-learning pass, iteration by iteration.

ℓ	1	2	3	4	5	6	7	8	9	10
k	0	1	2	3	0	1	2	3	0	1
x_k	1	0	2	2	0	0	1	0	0	1
u_k	1	2	0	–	1	2	1	–	1	1
d_k	2	0	0	–	1	1	2	–	0	1
$g(x_k, u_k, d_k)$	−11.1	10.9	2	–	−3.1	1.9	−11.1	–	5.9	−2.1
$Q_{\ell+1}(k, x_k, u_k)$	−5.55	5.45	−3	–	−1.55	3.675	−5.55	–	2.175	−3.825

At $\ell = 1$, with $x_1 = \max\{1 + 1 - 2, 0\} = 0$ and every estimate still zero,

$$Q_2(0,1,1) = 0.5\, g(1,1,2) + 0.5 \min_{v\in\{0,1,2\}} Q_1(1,0,v) = 0.5(1 + 5.9 - 18) = -5.55.$$

At $\ell = 2$, $x_2 = \max\{0 + 2 - 0, 0\} = 2$, where the only admissible control is $v = 0$:

$$Q_3(1,0,2) = 0.5\, g(0,2,0) + 0.5 \min_{v\in\{0\}} Q_2(2,2,v) = 0.5(0 + 10.9 - 0) = 5.45.$$

At $\ell = 3$ we are at $k = 2 = h - 1$, so the second update applies with $x_3 = 2$ and $g_3(2) = -8$:

$$Q_4(2,2,0) = 0.5\, g(2,0,0) + 0.5\, g_3(2) = 0.5(2) + 0.5(-8) = -3.$$

At $\ell = 4$ the process is in the terminal stage: nothing to update.
At $\ell = 5$, $x_1 = \max\{0 + 1 - 1, 0\} = 0$:

$$Q_6(0,0,1) = 0.5\, g(0,1,1) + 0.5 \min_{v\in\{0,1,2\}} Q_5(1,0,v) = 0.5(-3.1) = -1.55.$$

Note the minimisation. Iteration $\ell = 2$ pushed $Q(1,0,2)$ up to 5.45, but $Q(1,0,0)$ and $Q(1,0,1)$ are still 0, so the minimum over v is 0, not 5.45. An optimistic initialisation of the untried controls keeps them winning the min — which is exactly how the min drives exploration of the *bootstrap*, independently of how the data were generated.
At $\ell = 6$, $x_2 = \max\{0 + 2 - 1, 0\} = 1$, and this is the second visit to $(1,0,2)$, so the old estimate enters with weight $1 - \gamma = 0.5$:

$$Q_7(1,0,2) = 0.5\underbrace{Q_6(1,0,2)}_{=5.45} + 0.5\Big(\underbrace{g(0,2,1)}_{=1.9} + \min_{v\in\{0,1\}} Q_6(2,1,v)\Big) = 0.5(5.45) + 0.5(1.9 + 0) = 3.675.$$

At $\ell = 7$, again $k = h - 1$, with $x_3 = \max\{1 + 1 - 2, 0\} = 0$ and $g_3(0) = 0$:

$$Q_8(2,1,1) = 0.5\underbrace{g(1,1,2)}_{=-11.1} + 0.5\, g_3(0) = -5.55.$$

At $\ell = 8$ the process is terminal again: nothing to update.
At $\ell = 9$, $x_1 = \max\{0 + 1 - 0, 0\} = 1$, second visit to $(0,0,1)$:

$$Q_{10}(0,0,1) = 0.5\underbrace{Q_9(0,0,1)}_{=-1.55} + 0.5\Big(\underbrace{g(0,1,0)}_{=5.9} + \min_{v\in\{0,1\}} Q_9(1,1,v)\Big) = -0.775 + 2.95 = 2.175.$$

At $\ell = 10$, the successor state is not printed in the table but is determined by it: $x_{11} = x_{10} + u_{10} - d_{10} = 1 + 1 - 1 = 1$ at stage 2. Now the bootstrap is *not* zero — $\ell = 7$ pushed $Q(2,1,1)$ down to -5.55, while $Q(2,1,0)$ is still 0, so the minimum is -5.55:

$$Q_{11}(1,1,1) = 0.5\underbrace{g(1,1,1)}_{=-2.1} + 0.5 \min_{v\in\{0,1\}} Q_{10}(2,1,v) = 0.5(-2.1 - 5.55) = -3.825.$$

The pairs $(1,0,2)$ and $(0,0,1)$ were visited twice, the pairs $(0,1,1)$, $(2,1,1)$, $(2,2,0)$ and $(1,1,1)$ once. Collecting:

$$\begin{aligned} Q_{11}(0,0,1) &= 2.175 \quad & Q_{11}(0,1,1) &= -5.55 \\ Q_{11}(1,0,2) &= 3.675 \quad & Q_{11}(1,1,1) &= -3.825 \\ Q_{11}(2,1,1) &= -5.55 \quad & Q_{11}(2,2,0) &= -3 \end{aligned}$$

and $Q_{11}(k,i,u) = 0$ for every other triple.
Ten iterations have touched six of the eighteen state-action pairs, two of them twice, and the estimates they carry are still a long way from the true Q-factors. That is the honest picture of Q-learning on a small budget, and it is worth comparing against `A_10_StochInventoryControlQLearning.ipynb`, which runs the same problem to convergence and prints the exact answer alongside.

Problem 10.5 Suppose that we wish to sell a used car by finding an optimal policy for accepting offers. The car is exactly two years old, and the offer for cars of this model and age in the initial month of the sale follows a distribution modelled by a random variable w_0 taking values in $\{c_1, c_2, \dots, c_C\}$, with $\text{Prob}[w_0 = c_i] = p_i$ and $\sum_{i=1}^{C} p_i = 1 - p_0$, where p_0 is the probability that there is no offer. The car depreciates at a rate α every month after the initial one: if $\bar{x}_k$ is the offer at month k, then $\bar{x}_k/\alpha^k$ has the same distribution as w_0.

After ten years the residual value of the car is almost zero, so we maximise

$$\sum_{k=0}^{\infty} \alpha^k g(x_k, u_k),$$

where $u_k \in \{0, 1\}$ is the decision at time k either to sell ($u_k = 1$) or not ($u_k = 0$). The state x_k takes either a numerical value or one of two non-numerical values: $x_k = \text{N}$ if there is no offer, and $x_k = T$ if an offer has already been accepted. Otherwise $x_k = \bar{x}_k/\alpha^k \in \{c_1, \dots, c_C\}$ is the offer at month k rescaled, so that

$$x_k = w_{k-1} \;\text{ if } x_k \neq \text{N},\; x_{k-1} \neq T \;\text{ and } u_{k-1} = 0, \qquad x_k = T \;\text{ if } u_{k-1} = 1 \text{ or } x_{k-1} = T,$$

where the w_k are independent and identically distributed with the same distribution as w_0. The reward is $g(\text{N}, u_k) = 0$ and $g(T, u_k) = 0$ for every u_k, and

$$g(x_k, 1) = x_k, \qquad g(x_k, 0) = 0.$$

Suppose the model parameters are not available — in particular the probabilities p_i of the distribution of w_k — and we want to use a learning technique to compute the optimal policy. Show that the general Q-learning algorithm, with constant step size $\gamma^\ell = \bar{\alpha}$,

$$Q^{\ell+1}(x_k, 0) = (1 - \bar{\alpha})Q^\ell(x_k, 0) + \bar{\alpha}\,\alpha \max\{Q^\ell(x_{k+1}, 0), Q^\ell(x_{k+1}, 1)\},$$

simplifies greatly, and that all the Q-functions can be computed from scalars β_ℓ satisfying

$$\beta_{\ell+1} = (1 - \bar{\alpha})\beta_\ell + \bar{\alpha}\,\alpha \max\{\beta_\ell, x_{\ell+1}\}.$$

i Solution

Two observations do all the work.

The "sell" Q-factor is known in closed form and needs no learning. Selling at an offer x collects $g(x, 1) = x$ and sends the state to T, which is absorbing and pays nothing from then on. So

$$Q(x, 1) = x + \alpha \cdot 0 = x$$

for every numerical x, exactly, with no expectation and no unknown probabilities in it. Half of the table is free. (At $x = \text{N}$ selling is not available, and $Q(\text{N}, \cdot)$ is handled by the next observation.)

The "wait" Q-factor does not depend on the state. Waiting collects $g(x, 0) = 0$ and lands on $x_{k+1} = w_k$, whose distribution is the same whatever the current offer is — the offers are i.i.d. So

$$Q(x, 0) = \alpha\,\mathbb{E}\big[\max\{Q(w, 0), Q(w, 1)\}\big]$$

and the right-hand side contains no x at all. Write β for this common value, so that $Q(x, 0) = \beta$ for every x, including $x = \text{N}$, where waiting is the only option.

Substituting both into the stated update is now immediate. The bootstrap term is

$$\max\{Q^\ell(x_{k+1}, 0), Q^\ell(x_{k+1}, 1)\} = \max\{\beta_\ell, x_{k+1}\},$$

and since the left-hand side $Q^{\ell+1}(x_k, 0)$ is the same scalar $\beta_{\ell+1}$ whatever x_k was, the whole algorithm collapses to

$$\beta_{\ell+1} = (1-\bar{\alpha})\beta_\ell + \bar{\alpha}\,\alpha \max\{\beta_\ell, x_{\ell+1}\},$$

which is the claimed recursion. When the sampled month has no offer, $x_{\ell+1} = \mathrm{N}$ and selling is unavailable, so the max is just β_ℓ — the update then only shrinks β towards $\alpha\beta$, which is the correct effect of a month spent waiting.

Three things are worth drawing out.

What has been learned. The algorithm maintains **one number** instead of $2C$ Q-factors, and that number is the value of waiting. Because $Q(x, 1) = x$ is known, the greedy policy reads

$$\mu(x) = \arg\max_u Q(x, u) = \begin{cases} 1 & \text{if } x \geq \beta \\ 0 & \text{if } x < \beta, \end{cases}$$

so β *is* the selling threshold. Learning the scalar is learning the policy, and the threshold structure is not an assumption here but a consequence of the two observations above.

Why the collapse happened. Not because the problem is small, but because of two structural facts: the action "sell" is terminating with a known immediate reward, and the uncontrolled part of the dynamics is i.i.d., so the post-decision state carries no information. Either property alone would help; together they reduce the Q-table to a scalar.

What it is. The recursion is a Robbins–Monro scheme for the fixed point $\beta = \alpha\,\mathbb{E}[\max\{\beta, w\}]$, which is the Bellman equation of the problem written in one line. It is exactly what `A_10_stoppingtimeQLearning.ipynb` implements, and Unit 11's `A_11_stoppingtimeSimPI.ipynb` runs the same recursion with the max removed — a *fixed* threshold instead of the optimal one — which is the difference between policy iteration's two halves in a single example.

Additional exercises

Problem 10.6 Consider the following discounted cost optimal control problem

$$\min_{u_k = \mu_k(x_k)} \sum_{k=0}^{\infty} \mathbb{E}\left[\alpha^k g(x_k, u_k)\right], \qquad \text{Prob}[x_{k+1} = j \mid x_k = i,\ u_k = \ell] = P_{ji}(\ell),$$

where $\alpha = 0.9$ and $g(i, \ell) = i^2 + \ell^2$ for every $i \in \{1, 2, 3\}$ and $\ell \in \{1, 2\}$, and

$$P(1) = \begin{bmatrix} 0.3 & 0.4 & 0.8 \\ 0.7 & 0 & 0 \\ 0 & 0.6 & 0.2 \end{bmatrix}, \qquad P(2) = \begin{bmatrix} 0.1 & 0.5 & 0.6 \\ 0.9 & 0 & 0 \\ 0 & 0.5 & 0.4 \end{bmatrix}.$$

The initial condition is arbitrary, $x_0 \in \{1, 2, 3\}$. The optimal policy is $\mu(1) = 1$, $\mu(2) = 1$, $\mu(3) = 1$. Compute the optimal Q-function.

i Solution

As in Problem 10.3, begin from the Bellman equation with the optimal policy substituted. There is no terminal state now, so all three equations are coupled:

$$J(1) = g(1,1) + \alpha \sum_j P_{j1}(1) J(j) = 1^2 + 1^2 + 0.9(0.3J(1) + 0.7J(2))$$

$$J(2) = g(2,1) + \alpha \sum_j P_{j2}(1) J(j) = 2^2 + 1^2 + 0.9(0.4J(1) + 0.6J(3))$$

$$J(3) = g(3,1) + \alpha \sum_j P_{j3}(1) J(j) = 3^2 + 1^2 + 0.9(0.8J(1) + 0.2J(3))$$

This is a linear system $(I - \alpha P(1)^\top)J = g(\cdot, 1)$, whose solution is

$$\begin{bmatrix} J(1) \\ J(2) \\ J(3) \end{bmatrix} = \begin{bmatrix} 45.4735 \\ 49.5169 \\ 52.1231 \end{bmatrix}.$$

The optimal Q-function is $Q(i,u) = i^2 + u^2 + \alpha \sum_j P_{ji}(u) J(j)$. For $u = 1$:

$$Q(1,1) = 2 + \alpha \begin{bmatrix} 0.3 & 0.7 & 0 \end{bmatrix} J = 45.4735, \qquad Q(2,1) = 5 + \alpha \begin{bmatrix} 0.4 & 0 & 0.6 \end{bmatrix} J = 49.5169,$$

$$Q(3,1) = 10 + \alpha \begin{bmatrix} 0.8 & 0 & 0.2 \end{bmatrix} J = 52.1231,$$

and for $u = 2$:

$$Q(1,2) = 2 + \alpha \begin{bmatrix} 0.1 & 0.9 & 0 \end{bmatrix} J = 49.2013, \qquad Q(2,2) = 8 + \alpha \begin{bmatrix} 0.5 & 0 & 0.5 \end{bmatrix} J = 51.9185,$$

$$Q(3,2) = 13 + \alpha \begin{bmatrix} 0.6 & 0 & 0.4 \end{bmatrix} J = 56.3200.$$

At each state the optimal action is the minimiser of $Q(i,u)$, and $Q(i,1) < Q(i,2)$ for all three states, confirming the policy given in the problem statement. As in Problem 10.3, $Q(i,1) = J(i)$ for every i, because action 1 is the optimal one everywhere.
Note the row vectors above: each is a **column** of $P(u)$, written horizontally. For instance $\begin{bmatrix} 0.4 & 0 & 0.6 \end{bmatrix}$ is the second column of $P(1)$, the distribution of the next state given $x_k = 2$, $u_k = 1$.

Problem 10.7 Consider an inventory control problem where the number of items of a given product is described by

$$x_{k+1} = \max\{x_k + u_k - d_k, 0\}, \qquad k \in \{0, 1, \dots\},$$

where $x_k \in \{0, \dots, N\}$, $u_k \in \{0, \dots, N - x_k\}$ and d_k denote the number of items, the supply and the demand at time k, and N denotes the capacity. Suppose $N = 2$. The objective is to minimise

$$\mathbb{E}\left[\sum_{k=0}^{\infty} \alpha^k (c_1(x_k) + c_2(u_k) - p \min\{x_k + u_k, d_k\})\right]$$

for $\alpha = 0.9$, where $c_1(i) = i$ is the storage cost, $c_2(j) = 5j + 0.9$ for $j > 0$ and $c_2(0) = 0$ is the cost of ordering j items, and $p = 9$ is the selling price per item.

Suppose Q-learning is used to obtain the optimal policy. The estimate at iteration ℓ for state i using control $u \in \{0, \dots, N - i\}$ is denoted $Q_\ell(i,u)$ — there is no stage index now. A random policy with random initial conditions generates the data below. All estimates are initialised to zero.

ℓ	1	2	3	4	5	6	7	8	9	10
k	0	1	2	3	4	5	6	7	8	9
x_k	1	0	2	2	0	0	1	0	0	1
u_k	1	2	0	0	1	2	1	2	1	1
d_k	2	0	0	2	1	1	2	2	0	1

Provide the estimates $Q_9(i,u)$ for every i, u. Assume the step sizes are constant, $\gamma^\ell = 0.6$, in the update

$$Q_{\ell+1}(x_k,u_k)=(1-\gamma^\ell)Q_\ell(x_k,u_k)+\gamma^\ell\Big(g(x_k,u_k,d_k)+\alpha\min_{v\in U(x_{k+1})}Q_\ell(x_{k+1},v)\Big),$$

where $g(x_k,u_k,d_k)=c_1(x_k)+c_2(u_k)-p\min\{x_k+u_k,d_k\}$.

Solution

Three differences from Problem 10.4 are worth naming before computing. There is no stage index, so a single table $Q(i,u)$ of six entries is maintained rather than one table per stage. The discount factor appears explicitly in the bootstrap, where in 10.4 it was hidden in the finite horizon. And there is no terminal stage, so every row of the data uses the same update — the single row of the table is one long trajectory, not three short ones. (The successor states are consistent with it: $\max\{x_k+u_k-d_k,0\}$ always equals the next column's x_{k+1}.)
Replaying the ten updates:

Table 2: Ten Q-learning updates. Each row's last column replaces the previous estimate of that pair.

ℓ	x_k	u_k	d_k	$g(x_k,u_k,d_k)$	x_{k+1}	$\min_v Q_\ell(x_{k+1},v)$	$Q_{\ell+1}(x_k,u_k)$
1	1	1	2	-11.1	0	0	$Q(1,1)=-6.66$
2	0	2	0	10.9	2	0	$Q(0,2)=6.54$
3	2	0	0	2	2	0	$Q(2,0)=1.20$
4	2	0	2	-16	0	0	$Q(2,0)=-9.12$
5	0	1	1	-3.1	0	0	$Q(0,1)=-1.86$
6	0	2	1	1.9	1	-6.66	$Q(0,2)=0.1596$
7	1	1	2	-11.1	0	-1.86	$Q(1,1)=-10.3284$
8	0	2	2	-7.1	0	-1.86	$Q(0,2)=-5.2006$
9	0	1	0	5.9	1	-10.3284	$Q(0,1)=-2.7813$
10	1	1	1	-2.1	1	-10.3284	$Q(1,1)=-10.9687$

A sample, for the arithmetic. At $\ell = 6$ the state-action pair $(0, 2)$ is revisited, so its old estimate enters with weight $1 - \gamma = 0.4$, and the successor $x_6 = \max\{0 + 2 - 1, 0\} = 1$ has $Q_6(1, 1) = -6.66$ from $\ell = 1$ against $Q_6(1, 0) = 0$, so the minimum is -6.66:

$$Q_7(0, 2) = 0.4(6.54) + 0.6(1.9 + 0.9(-6.66)) = 2.616 + 0.6(-4.094) = 0.1596.$$

At $\ell = 4$ the pair $(2, 0)$ is revisited one step after being set, and the demand $d = 2$ makes the running cost $2 - 9 \cdot 2 = -16$, so the estimate swings from $+1.20$ to -9.12 in a single update. A step size of 0.6 keeps only 40% of what was learned, which is why these early estimates move so violently.

The answer, and a word on the indexing. The question asks for "the estimates after 9 iterations, $Q_9(i, u)$", and those are two different things, because Q_ℓ is the estimate *before* update ℓ. Taken literally, Q_9 is the result of updates $\ell = 1, \ldots, 8$:

$$Q_9(0, 1) = -1.86, \quad Q_9(0, 2) = -5.2006, \quad Q_9(1, 1) = -10.3284, \quad Q_9(2, 0) = -9.12,$$

with $Q_9(0, 0) = Q_9(1, 0) = 0$. After all ten updates, matching the convention of Problem 10.4, where $L = 10$ iterations gave Q_{11}:

$$Q_{11}(0, 1) = -2.7813, \quad Q_{11}(0, 2) = -5.2006, \quad Q_{11}(1, 1) = -10.9687, \quad Q_{11}(2, 0) = -9.12.$$

Only the two pairs touched at $\ell = 9, 10$ differ between the two readings.

Two pairs, $(0, 0)$ and $(1, 0)$ — order nothing, from an empty or a single-item shop — are never visited, so they keep their initial value of zero. Since the true Q-factors here are all negative (the shop makes money), a stored zero is an *over*estimate of a cost, so those two actions would never be chosen by a greedy policy, and the data would never correct them. That is the exploration problem of Unit 10 in its smallest form, and the same effect is visible — with real numbers — in the unvisited high-stock entries of `A_11_StochInventoryControlSimBasedPI.ipynb`.